C++ Programming Individual Assignment

Student Course Management System

Code Documentation Report

Student Name	عمرو جمال حسن عبدالسميع
Military Number	11189
GitHub Repository	https://github.com/amr0603/st
Programming Language	C++
Assignment Type	Individual Assignment

1. Program Overview

The Student Course Management System is a C++ console application designed to manage student records and their course enrollments. The system is built using core Object-Oriented Programming (OOP) principles combined with Standard Template Library (STL) containers and algorithms.

The application provides a fully interactive menu-driven interface allowing users to perform all essential academic management operations.

1.1 Objectives

- Design and implement classes using OOP principles (Encapsulation, Inheritance, Polymorphism)
- Use STL containers (vector, set) to manage collections of data
- Implement searching and sorting algorithms from the STL
- Build a modular, well-structured console application

1.2 Included Libraries

Library	Purpose
`#include <iostream>`	Standard input/output operations (cin, cout)
`#include <vector>`	Dynamic array container for storing students
`#include <string>`	String class and manipulation functions
`#include <algorithm>`	STL algorithms: sort(), find_if(), remove_if()
`#include <set>`	Ordered set container for unique course names

2. Class Design & OOP Principles

2.1 Class Hierarchy

The program uses two classes with an inheritance relationship:

```
[ person ]   (Base Class)
-------------------------------
        # name : string
-------------------------------
     + person(n: string)
  + setName(n: string) : void
     + getName() : string
  + display() : void  [virtual]
```

```
                    ▲  (Inheritance)
                       |
[ student ]   (Derived Class)
──────────────────────────────────
            - id   : int
           - gpa  : float
      - courses : set<string>
──────────────────────────────────
         + student(i, n, g)
     + addCourse(courseName) : void
     + display() : void  [override]
  + getId(), getGpa(), setId(), setGpa()
```

2.2 Base Class: person

The person class serves as the base class and encapsulates the common attribute name that is shared across different types of people in the system.

Key Features:

- Protected member name — accessible by derived classes but not publicly exposed
- Default constructor with optional name parameter
- Getter/setter for name attribute
- Virtual display() method enabling polymorphism

Code Snippet — person class

```
class person {
protected:
    string name;
public:
    person(string n = "") { name = n; }
    void setName(string n) { name = n; }
    string getName() const { return name; }
    virtual void display() const {
        cout << "Name: " << name << endl;
    }
};
```

2.3 Derived Class: student

The student class inherits from person and extends it with additional attributes and behaviour specific to a student. It uses OOP encapsulation to keep all attributes private and exposes them only through public getter/setter methods.

Attribute	Type	Description
`id`	`int`	Unique student identifier

`gpa`	`float`	Grade Point Average (0.0 – 4.0)
`courses`	`set<string>`	Set of enrolled course names (no duplicates)
`name (inherited)`	`string`	Student's full name (from person base class)

2.4 OOP Concepts Applied

OOP Concept	How It Is Applied
Encapsulation	All student attributes (id, gpa, courses) are private. Access is controlled via public getters and setters only.
Inheritance	student inherits from person using public inheritance. The name attribute and getName()/setName() methods are reused without duplication.
Polymorphism	display() is declared virtual in person and overridden in student, allowing runtime polymorphism.
Constructor	person uses a default-value constructor. student uses a parameterized constructor with an initializer list (person(n)).
Abstraction	The person class abstracts the concept of a named entity, while student adds domain-specific details.

3. STL Containers & Algorithms

3.1 STL Containers Used

Container 1: vector<student>

A dynamic array storing all student objects. Declared in main() as the primary data store.

```
vector<student> students;
```

Operation	Usage in Code
`push_back()`	Adds a new student to the list
`erase()`	Removes a student from the vector after remove_if()
`empty()`	Checks if no students exist before display/sort
`begin() / end()`	Used with algorithms (find_if, sort, remove_if)

Container 2: set<string> (courses inside student)

Each student object contains a set<string> to store enrolled course names. The set automatically prevents duplicate entries and maintains alphabetical order.

```
set<string> courses;
```

```
void addCourse(string courseName) {
    auto result = courses.insert(courseName);
    if (!result.second) {
        cout << "Student is already enrolled in this course!\n";
    }
}
```

The insert() method returns a pair<iterator, bool>. The bool (result.second) is false if the course already exists, enabling duplicate detection without extra logic.

3.2 STL Algorithms Used

Algorithm	Used For	Menu Option
`find_if()`	Search for student by ID; also check for duplicate IDs on add	Option 1, 3, 5, 6
`sort()`	Sort students by GPA from highest to lowest	Option 4
`remove_if()`	Mark student for removal by ID (used with erase())	Option 6

4. Menu Functions — Detailed Explanation

The main() function runs a do-while loop presenting a menu of 7 options. Below is a detailed explanation of each option.

Option 1 — Add Student

Prompts the user for student ID, name, and GPA. Validates:

- ID uniqueness: uses find_if() to check the vector for an existing student with the same ID
- GPA range: checks that GPA is not below 0.0 (note: full upper-bound check of > 4.0 is a suggested improvement — see Section 6)

```
auto it = find_if(students.begin(), students.end(), [id](const student& s) {
    return s.getId() == id;
});
if (it != students.end()) {
    cout << "Error: Student ID already exists!\n";
    continue;
```

```
}
```

Option 2 — Display All Students

Iterates over the students vector and calls display() on each student object. If the vector is empty, a message is shown instead. The display() method prints ID, name, GPA, and all enrolled courses.

Option 3 — Search Student by ID

Uses find_if() with a lambda function to locate a student by their ID. If found, the student's full profile is displayed. If not found, a not-found message is shown.

```
auto it = find_if(students.begin(), students.end(), [searchId](const student& s) {
    return s.getId() == searchId;
});
```

Option 4 — Sort Students by GPA

Applies the STL sort() algorithm with a custom comparator (lambda) to arrange students in descending order of GPA (highest GPA first).

```
sort(students.begin(), students.end(), [](const student& a, const student& b) {
    return a.getGpa() > b.getGpa();
});
```

Option 5 — Add Course to Student

Locates the student by ID using find_if(). If found, prompts for a course name and calls addCourse() on the student object. The set<string> container ensures no duplicate courses are added for the same student.

Option 6 — Delete Student

Uses remove_if() combined with erase() (the erase-remove idiom) to delete a student by ID. remove_if() moves the matching element to the end of the vector and returns an iterator to it; erase() then removes it permanently.

```
auto it = remove_if(students.begin(), students.end(), [deleteId](const student& s) {
    return s.getId() == deleteId;
});
if (it != students.end()) {
    students.erase(it, students.end());
    cout << "Student deleted successfully!\n";
}
```

Option 7 — Exit

Prints a farewell message and exits the do-while loop by setting choice == 7, terminating the program cleanly.

5. Sample Input / Output

Below is a sample interaction demonstrating the core features of the system.

Sample Input

```
Choice: 1
Enter ID: 1001
Enter Name: Ahmed Ali
Enter GPA: 3.8

Choice: 5
Enter Student ID: 1001
Enter Course: Data Structures

Choice: 3
Enter ID to search: 1001

Choice: 4

Choice: 7
```

Expected Output

```
Student added successfully!

Course added successfully!

Student Found
ID: 1001 | Name: Ahmed Ali | GPA: 3.8
Courses: [Data Structures]
-----------------------

Students sorted by GPA (Highest to
Lowest)!

Exiting system. Goodbye!
```

6. Input Validation & Error Handling

Validation Case	Handling Method	Status
Duplicate Student ID	find_if() checks before adding	**Implemented**
GPA below 0.0	Rejected with error message	**Implemented**
GPA above 4.0	Not currently validated	**Suggested Fix**
Duplicate Course	set<string> prevents duplicates automatically	**Implemented**
Empty student list	Checked before display/sort	**Implemented**
Student not found	Error message printed for search/delete	**Implemented**
Invalid menu choice	Default else branch handles unknown input	**Implemented**

7. Assignment Requirements Compliance

Requirement	Section	Status
Classes with encapsulation	Section 2.2 / 2.3	✔
At least one inheritance example	Section 2.1	✔
Constructors	Section 2.4	✔
Virtual function / Polymorphism	Section 2.4	✔
STL vector container	Section 3.1	✔
STL set container (bonus)	Section 3.1	✔
sort() algorithm	Section 3.2	✔
find_if() algorithm	Section 3.2	✔
remove_if() algorithm	Section 3.2	✔
Add / Remove / Search / Display students	Section 4	✔
Enroll student in course	Section 4 – Option 5	✔
Sort students by GPA	Section 4 – Option 4	✔
Unique student IDs	Section 6	✔
GPA constraint (0.0 – 4.0)	Section 6	**Partial**
No duplicate courses per student	Section 3.1	✔
Bonus: set for course uniqueness	Section 3.1	✔

8. Conclusion

This Student Course Management System successfully demonstrates the application of fundamental C++ programming concepts as required by the assignment. The program implements:

- A two-class OOP hierarchy (person and student) with full encapsulation, inheritance, and polymorphism
- Two STL containers (vector<student> and set<string>) used purposefully for their respective data management roles
- Three STL algorithms (sort, find_if, remove_if) applied with lambda expressions for clean and concise logic
- A complete menu-driven interface covering all six functional requirements
- Input validation and duplicate detection to ensure data integrity

The only minor gap identified is the upper-bound GPA validation (gpa > 4.0), which is not currently enforced. Adding this check would make the system fully compliant with all constraints.

> **GitHub Repository:** https://github.com/amr0603/st